

SQL Row-Level Functions

4 · Row-Level Functions — Complete Reference Guide (Banking Project)

Table of Contents

1. String
2. Number
3. Date & Time
4. Null Functions
5. Case Statements

1. String

→ **Task 1: Display the full name of each customer in a single field by merging their first and last names in uppercase (Handle Text Formatting).**

Additionally, generate a short username using the first letter of the first name plus the last name in lowercase, and provide the length of each first name.

```
SELECT
    customer_id,
    UPPER(CONCAT(TRIM(first_name), ' ', TRIM(last_name)))
        AS clean_full_name,
    LOWER(CONCAT(LEFT(first_name, 1), last_name))
        AS username_style,
    LEN(TRIM(first_name)) AS first_name_length
FROM Customers;
```

→ **Task 2: Update each transaction description by replacing the word 'Cash' with 'CASH' (Modify Text).**

Additionally, extract the first 20 characters and the last 10 characters of each description.

```
SELECT
    transaction_id, description,
    REPLACE(description, 'Cash', 'CASH') AS description_updated,
    SUBSTRING(description, 1, 20) AS first_20_chars,
    RIGHT(description, 10) AS last_10_chars
FROM Transactions;
```

2. Number

→ **Task 3: Round every account balance to the nearest hundred (Handle Numeric Rounding).**

```
SELECT
    account_id, account_balance,
    ROUND(account_balance, -2) AS rounded_to_nearest_100
FROM Accounts;
```

→ **Task 4: Round each account's interest rate to 1 decimal place (Handle Numeric Rounding).**

```
SELECT
    account_id, interest_rate,
    ROUND(interest_rate, 1) AS rounded_interest_rate
FROM Accounts;
```

3. Date & Time

→ **Task 5: Extract the day, month, year, month name, and quarter from each transaction date, along with the last date of that month (Extract Date Parts).**

```
SELECT
    transaction_id, transaction_date,
    DAY(transaction_date) AS txn_day,
    MONTH(transaction_date) AS txn_month,
    YEAR(transaction_date) AS txn_year,
    DATENAME(MONTH, transaction_date) AS month_name,
    DATEPART(QUARTER, transaction_date) AS txn_quarter,
    EOMONTH(transaction_date) AS month_end_date
FROM Transactions;
```

→ **Task 6: Calculate the number of days since each account was opened and the date 30 days after opening (Date Calculations).**

Additionally, find the start of that month, and format, convert, cast, and validate the opening date.

```
SELECT
    account_id, open_date,
    DATEDIFF(DAY, open_date, GETDATE()) AS days_since_opening,
    DATEADD(DAY, 30, open_date) AS grace_period_end_date,
    DATETRUNC(MONTH, open_date) AS start_of_that_month,
    FORMAT(open_date, 'dd-MMM-yyyy') AS nicely_formatted_date,
    CONVERT(VARCHAR, open_date, 103) AS converted_date,
    CAST(open_date AS DATETIME) AS cast_as_datetime,
    ISDATE(CONVERT(VARCHAR, open_date)) AS is_this_a_valid_date
FROM Accounts;
```

4. Null Functions

→ **Task 7: Find accounts with a missing interest rate and replace it with 0, using both ISNULL and COALESCE (Handle Null Values).**

Additionally, provide details such as Account ID and Account Type.

```
SELECT
    account_id, account_type, interest_rate,
    ISNULL(interest_rate, 0) AS isnull_version,
    COALESCE(interest_rate, 0) AS coalesce_version
FROM Accounts
WHERE interest_rate IS NULL;
```

→ **Task 8: Flag transactions of exactly 1000 by converting the amount to NULL (Handle Null Values Using NULLIF).**

Additionally, filter the results to only transactions where the merchant category is known.

```
SELECT
    transaction_id, transaction_amount, merchant_category,
    NULLIF(transaction_amount, 1000) AS amount_unless_exactly_1000
FROM Transactions
WHERE merchant_category IS NOT NULL;
```

5. Case Statements

→ **Task 9: Generate a report categorizing each account by balance (Categorize Data):**

Balance > 100000 : High Balance

Balance Between 10000 and 100000 : Medium Balance

Balance < 10000 : Low Balance

Sort the result from lowest to highest balance.

```
SELECT
    account_id, account_balance,
    ROUND(account_balance, -3) AS rounded_balance,
    CASE
        WHEN ROUND(account_balance, -3) < 10000
            THEN 'Low Balance'
        WHEN ROUND(account_balance, -3) BETWEEN 10000 AND 100000
            THEN 'Medium Balance'
        ELSE 'High Balance'
    END AS balance_category
FROM Accounts;
```

→ **Task 10: Label each transaction as MONEY IN, MONEY OUT, or TRANSFER based on its transaction type, and display the label in uppercase (Categorize Data).**

```
SELECT
    transaction_id, transaction_type,
    UPPER(
        CASE
            WHEN transaction_type IN ('Deposit', 'Interest Credit')
                THEN 'money in'
            WHEN transaction_type IN ('Withdrawal', 'Fee')
                THEN 'money out'
            WHEN transaction_type = 'Transfer' THEN 'transfer'
        END
    ) AS plain_english_tag
FROM Transactions;
```